PHASE 1 + PHASE 2

Manufacturing ERP and Autonomous Fulfilment, explained simply

A plain-English guide to the technology stack, system architecture, data flow, security, AI controls, hosting model, and the practical difference between both phases.

PHASE 1 · FOUNDATION

The ERP

Stores and runs the company's orders, products, BOMs, inventory, purchasing, production, quality, accounts, people, maintenance, approvals and audit records.

PHASE 2 · COORDINATION

The mission controller

Takes a confirmed customer order, gathers evidence, compares fulfilment plans, checks authority, records actions, watches for changes and can replan.

Same core system. A new control layer.

Phase 2 is not a rewrite and not a set of microservices. It extends Phase 1's TypeScript modular monolith and PostgreSQL database with durable fulfilment missions.

ARCHITECTURE

Boundary-enforced modular monolith
One backend, cleanly separated business modules.

SAFETY RULE

Code decides; AI explains
Models do not own accounting, stock or policy verdicts.

CURRENT MATURITY

Advanced prototype
Real ERP data and mission records; some execution remains simulated.

SHARED FOUNDATION

1. Technology stack used by both phases

Phase 2 keeps the Phase 1 stack. The change is mainly new orchestration logic, database records, API routes and Mission Control screens—not a new technology platform.

LAYER	TECHNOLOGY	WHAT IT DOES
Language & runtime	TypeScript 5.7 · Node.js 22	One strongly typed language across the browser, API and shared rules.
Frontend	Next.js 15 · React 19	Builds the screens, navigation and browser experience.
Interface design	Tailwind 4 · Radix UI · Lucide	Provides layouts, controls, accessibility primitives and icons.
Backend	NestJS 11 · Express 5 · Zod	Receives requests, validates data and runs ERP business services.
Database	PostgreSQL 17 · Drizzle ORM	Stores permanent business, audit, approval and mission records.
Search / AI data	pgvector · pg_trgm	Supports similarity, search and future retrieval use cases.
Identity & access	Keycloak 26 · OIDC · RBAC	Signs users in and checks which operations they may perform.
Tenant isolation	PostgreSQL FORCE RLS	Keeps each company's rows separated inside the database.
Jobs & events	Valkey 8 · BullMQ · outbox	Runs retryable background work without losing database events.
Documents & AI	Gotenberg · stub / Ollama	Packages PDF rendering; offers offline deterministic or optional local AI.
Build & hosting	pnpm · Docker · Vercel adapters	Manages the monorepo, local services and demo deployments.

Repository shape

```
XELOR/
├─ apps/
│  ├─ web      Next.js browser app
│  ├─ api      NestJS API + worker
│  └─ edge     factory simulator boundary
├─ packages/
│  ├─ platform pure business rules
│  └─ db       schema, migrations, RLS
└─ infra/      Docker and deployment files
```

What "modular monolith" means

Sales, Purchase, Inventory, Production, Quality and Accounts are separate modules, but they run together as one main backend and use one primary PostgreSQL database.

Modules exchange information through defined interfaces or events. Code-boundary checks help stop one department from reaching directly into another department's internals.

This is easier to deploy than many microservices while keeping business ownership clear.

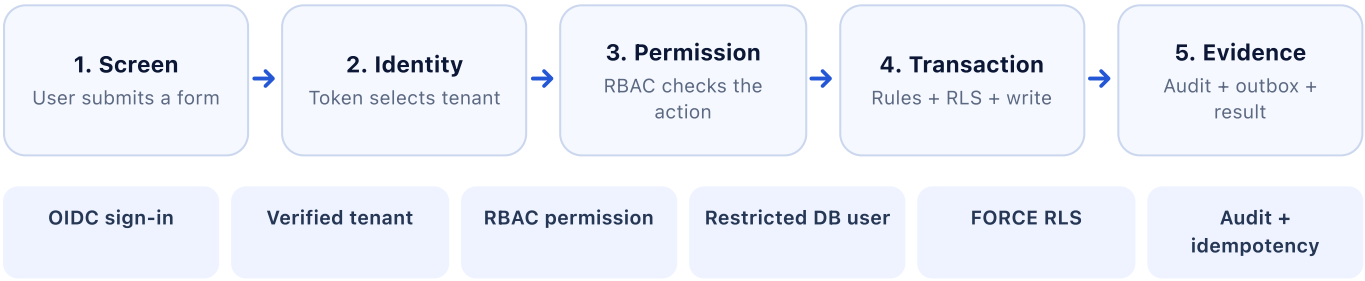
Important architectural choice: exact business calculations—GST, MRP, inventory, accounting, payroll, quality, scrap and policy gates—live in deterministic code. A language model may explain a result, but it does not replace the calculation.

PHASE 1

2. ERP foundation and control architecture

Phase 1 is the system of record: it owns business documents, calculations, permissions, approvals and ledgers. The browser never talks directly to the database.

A normal write request



Reliability

Business write, audit record and outbox event commit together.

Idempotency keys prevent a network retry from creating the same document twice.

AI and Agent OS

One governed AI router checks registry, opt-out, kill switch, budget and logging.

Agents run bounded graphs with evidence, checkpoints and human approval gates.

Honest boundary

Default AI is deterministic; Ollama is optional. No hosted LLM adapter is active.

The edge app and external industrial connections remain simulation boundaries.

Local runtime

Phase 1: web :3001 · API :3000 · PostgreSQL :5432 · Valkey :6379 · Keycloak :8080 · Gotenberg :3002.

Phase 2 workspace override: web :3101 · API :3100 with a separate database so both phases can run together. Repository defaults remain :3001 / :3000.

Deployment position

Packaging exists for Vercel, Railway and single-host Docker; this does not mean all three are operating. Deployment observed on 11 August 2026 used Vercel and managed PostgreSQL in Singapore.

The intended AWS Mumbai production platform remains planned, not commissioned.

Public-demo warning: the fixed demo-identity bypass is acceptable only for isolated synthetic data. Real customer data requires normal production identity and controls.

PHASE 2

3. Policy-governed autonomous fulfilment

Phase 2 adds a durable mission above the ERP. A confirmed order can now carry its objective, evidence, plan versions, approval, actions, events and outcome over time.

Observe › **Diagnose** › **Plan** › **Authorize** › **Act** › **Verify** › **Wait / Replan**

1–2 · Accept & engineer
Read the order and prove one active BOM exists.

3–5 · Gather evidence
Calculate shortages, capacity and qualified supply.

6–7 · Decide & critique
Compare strategies and recompute the winner from live facts, derived calculations and labelled seeded assumptions.

8 · Authorize
Apply autonomy limits or stop for a plan-specific approval.

9–11 · Controlled action
Reserve stock, create supplier-specific draft POs and create pegged planned production orders.

12–13 · Watch & close
React to events, replan when required and verify the result.

Deterministic planner

Generates only strategies supported by current evidence: stock-only, normal supplier, faster expedite or split sourcing.

Calculates material date, production time, schedule slack, total cost, margin, premium, supplier reliability, capacity confidence and policy breaches.

A physical impossibility cannot be approved away; a feasible policy exception can be escalated.

Five responsibility planes

- **Mission:** objective, state, dependencies and completion.
- **Decision:** evidence, candidates and bounded critique.
- **Truth:** ERP facts and deterministic calculations.
- **Governance:** authority, policy, approval and risk.
- **Execution:** controlled action, observation and verification.

New durable mission records

Mission
Customer promise and definition of done.

Plan version
Frozen strategy with digest and expected result.

Step
Evidence, finding, explanation and status.

Action
Proposed change, execution and separate verification.

Event
Something changed, plus its mission impact.

Approval
Human authority tied to one exact plan digest.

Trace spine: Phase 2 connects the sales-order line to reservation, purchase requisition, PO line, planned order, production order and quality release. It also enforces one active BOM per item and strengthens scrap and quality-disposition rules.

Mission Control experience

Confirmed orders can start a mission; users select an A2/A3/A4 autonomy envelope. The stream repeatedly calls the one-step `advance()` operation, allowing users to watch the 13-step timeline, inspect live/derived/seeded evidence, compare candidate plans, decide approval cards, simulate a supplier delay and review the final outcome. The UI now contains 24 installed modules grouped into seven job-based workspaces.

ASSESSMENT

4. Phase comparison and current readiness

QUESTION	PHASE 1	PHASE 2
Main job	Run and record business departments.	Coordinate one order across those departments.
Role	System of record.	Mission and control layer.
Planning	MRP and departmental worklists.	Cross-functional strategy, approval and replanning.
Data	ERP documents, ledgers and audit.	Adds missions, plans, steps, actions, events and approvals.
Agents	Bounded reviews and governed work items.	Durable fulfilment mission with wait/replan intent.
Maturity	Broad guarded ERP prototype.	Advanced orchestration and investor prototype.

Implemented and backed by real records

- ✓ Durable state and at most one active mission per order
- ✓ Live ERP evidence reads and deterministic planning
- ✓ Cost, date, margin, risk and feasibility comparison
- ✓ Immutable plan versions and digest-bound approvals
- ✓ Step, action, event and postcondition history
- ✓ Per-line, partial-stock demand and trace pegs
- ✓ Real draft POs and planned production orders
- ✓ Retry-safe actions and a durable waiting boundary
- ✓ Closure checks dispatch, quality, invoice and margin evidence

Still simulated or incomplete

- ! Purchasing creates draft POs directly; no PR workflow is created
- ! No event bridge yet wakes a waiting mission from downstream ERP changes
- ! PO approval, receipt and production completion are not autonomously progressed
- ! Actual production cost and actual margin roll-up are not available
- ! Dispatch proves ERP shipment, not external customer receipt
- ! No database-backed test yet proves the complete order-to-delivery chain
- ! Concurrent missions could over-promise shared stock
- ! Supplier and physical-world events are seeded simulations

Highest-priority next work

1 · Wake safely
Connect real ERP events to impact analysis and mission resume.

2 · Progress
Drive PO approval, receipt and production completion through governed capabilities.

3 · Cost
Roll up actual production cost and calculate actual margin.

4 · Protect ATP
Lock and net stock across all competing reservations.

5 · Test
Run one database-backed order through the full document chain.

Final assessment

Phase 1 is a well-guarded manufacturing ERP foundation. Phase 2 adds a strong, traceable and approval-aware coordination layer above it. The architectural direction is sound, but Phase 2 should currently be presented as an **autonomous-fulfilment prototype**, not yet as proven end-to-end autonomous delivery.

Evidence base: PHASE-1/README.md · apps/api/src/app.module.ts · packages/db/src/client.ts · infra/docker-compose.yml · PHASE-2/UPGRADE.md · apps/api/src/fulfilment/mission.service.ts · packages/platform/src/fulfilment/planner.ts · migrations 0092–0097 · Mission Control UI.